

myCompiler

Small C compiler for LLVM IR

一、專案概述

以 ANTLR 4.13.2 為語法分析框架、Java 為實作語言，開發一個可以將 C 語言子集翻譯成 LLVM IR 的編譯器。產生的 .ll 檔案可以透過 clang 搭配隨附的執行期函式庫 myRuntime.c 編譯成可執行檔。

二、繳交檔案結構

檔案 / 目錄	說明
myCompiler.g4	ANTLR 4 語法檔：Lexer 預處理器 + Parser + 程式碼產生邏輯
myCompiler_test.java	程式進入點：讀取 .c 檔、呼叫預處理器、Parser、輸出 LLVM IR
myRuntime.c	執行期函式庫，提供 ## 運算子底層實作（my_hashhash）
Makefile	一鍵建置、執行全部測試、回歸比對
antlr-4.13.2-complete.jar	ANTLR 執行期 JAR，已隨專案附上
test/（目錄）	所有測試 .c 檔及 stdin 輸入 (*.in)
expected/（目錄）	各測試預期輸出，用於 make check 自動比對
*.ll	測試檔產生的.ll 檔案
README	myCompiler 功能詳細說明及編譯測試方法
myCompiler_Organized_Extension_List	myCompiler 擴充功能說明以及對應測試檔案
C_Subset	myCompiler C subset 概述

三、建置與執行方法

3.1 環境需求

- Java JDK 11 以上
- Clang（含 LLVM 後端）
- GNU make
- gcc（選用，僅 make gen_expected 時需要）

3.2 常用指令(假設檔名為 test1.c)

指令	說明
make	產生 Parser → 編譯 Java → 執行所有測試
make run TEST=test1	編譯單一 .c 並執行 (若有 .in 自動餵給 stdin)
make ll TEST=test1	只產生 .ll，不連結不執行
make check (新增，見備註)	全部自動測試對比預期輸出，印出 PASS/FAIL
make check DIFF=1	同上，FAIL 時顯示完整 diff
make check1 TEST=test1	測試對比 test1 預期輸出
make gen_expected	用 gcc 跑全部測試，產生 expected/*.txt
make gen_expected1 TEST=test1	產生單一預期輸出 (用 gcc)
make clean	刪除 .class、ANTLR 產生 Java 原始檔、.tokens、測試二進位檔等中間產物
make clean_all	在 make clean 基礎上，額外清除 .interp 與 .ll

註：

1. 檔名請勿訂為 test.c，因為上層為 test/ 資料夾。
2. .in 檔案已附上；測試時請盡量測試正常的 C 輸入情形；有擴充 warning 情形。
3. make gen_expected 只需在一開始執行，之後 make check 即可，無需每次 gen。(已 gen 好)
4. 本編譯器具備語意分析與防呆攔截機制。依循系統標準，編譯錯誤會輸出至標準錯誤管道 (stderr)。另外，make check 在 FAIL 時對 diff 內容與 compiler stderr 摘要使用 head 限制行數（避免輸出過長），其他指令 (run/check1) 的 [Compiler STDERR] 為完整輸出。因此，在執行 make check 時，專門用於驗證錯誤處理的 test_semantic、test_bounds_dynamic 與 test_boundary 顯示為 FAIL 屬於正常現象，這代表編譯器的安全防護已成功觸發並攔截非法操作。test_new_features_v4 因為 compile 時間不同；test_new_easy 因為記憶體位置不同會 FAIL。(正常)
5. 測試 test_argc_funcptr.c 可使用 make run TEST=test_argc_funcptr ARGS="hello world 123" 指令；test_fileio.c 輸出後可使用 cat my_test_data.txt 觀看輸出內容(需要有寫入權限)；test_include 要把 test_helper.h 放在同目錄；test_multifile.deps、math_utils (顯示 clang 無法編譯 .ll -> SKIP 正常，只是輔助檔案) 要放進 test/ 目錄才能與 test_multifile.c 一起編譯。
6. 浮點數輸出可能因編譯器 float/double 精度略有差異。

四、C Subset

4.1 資料型別

型別	說明
整數	int、short、long / long long (i64)、bool / _Bool、char

型別	說明
unsigned 系列	unsigned char / short / int / long
浮點數	float(後綴 f)、double
空型別	void (限函式回傳值與 void* 指標)
固定寬度	int8_t ... int64_t、uint8_t ... uint64_t、size_t、time_t、clock_t、ptrdiff_t、ssize_t、off_t、FILE、va_list (不需 #include)
複合型別	struct (含位元欄位)、union、enum、typedef (含函式指標別名)
陣列	一維、二維、三維、VLA
指標	int*、int**...
函式指標	int (*fp)(int, int) 語法；typedef int (*Cmp)(int, int) 別名；全域與區域皆支援
複合字面值	(Type){init} 語法；動態建立匿名 struct 或陣列並取得指標；可用在運算式或函式引數中
typeof(expr)	GNU C 擴充；編譯期推導運算式型別，支援泛型巨集 (如 SWAP)；以 Dry-Run 模式推導，不產生 IR 副作用
_Alignof(type)	回傳型別在 x86_64 下的記憶體對齊位元組數；編譯期常數
匿名 struct / union	C11 標準匿名巢狀 struct / union；內部欄位可跨層直接存取 (如 v.x、col.r)，無需指定中間層名稱
_Static_assert	C11 編譯期靜態斷言；條件不成立時直接編譯錯誤 (不產生執行期程式碼)。例： _Static_assert(sizeof(int)==4, "int must be 4 bytes");
彈性陣列成員	C99：struct 最後一個欄位可宣告為未定長度陣列 (如 char data[];)。配合 malloc(sizeof(struct X) + N) 實現連續可變長記憶體佈局，零額外開銷
extern 符號	extern int x; 宣告外部全域變數；extern void foo(); 宣告外部函式。可與其他 .c 檔案分離編譯，透過 clang / ld 連結器跨檔案解析符號

4.2 運算子

運算子	說明
算術	+ - * / %；自訂 ## 運算子 (僅 float)
比較	> >= < <= == !=
邏輯	&& ! (含短路求值)

運算子	說明
位元	& ^ ~ << >> (及複合賦值 &= = ^= <<= >>=)
賦值	= += -= *= /= %= (變數或 struct 成員 ; arr[i] 只支援 =)
遞增遞減	++ -- (限變數 ID)
三元	? : (可巢狀)
sizeof	sizeof(型別) / sizeof(運算式) · 編譯期計算 ; _Alignof(type) 回傳對齊位元組數
逗號	常見用途為 for 的 update (如 i++, j--)
LLVM Intrinsics	__builtin_clz[l/ll] / __builtin_ctz[l/ll] / __builtin_popcount[l/ll] / __builtin_bswap16 / __builtin_bswap32 / __builtin_bswap64 ; 直接對應 LLVM llvm.ctlz / llvm.cttz / llvm.ctpop / llvm.bswap 底層指令

4.3 陳述式

陳述式	說明
變數宣告	int a=1, b=2; (同行同型別) ; const int N=10; ; static 局部變數
具名初始化	struct : { .x=10, .y=20 } ; 陣列 : {[2]=9, [4]=42} ; 未指定欄位自動為 0
if / else	任意巢狀 ; 常數條件編譯期消除
while / do-while / for	任意混合巢狀 ; for 支援 C99 宣告與多變數
switch	switch(整數) { case n: ... break; default: ... } ; 支援 fall-through
break / continue	break 跳出迴圈或 switch ; continue 只能用在迴圈內
goto	含前向 / 後向跳轉
return	return expr; 或 return; (void)
printf / scanf	格式字串必須是字面值 (詳見 5.1 節)
邊界防護	陣列存取自動靜態分析 (常數索引越界發出 Warning) 與動態注入 (執行期越界觸發 abort())
遺漏回傳值	非 void 函式若有路徑缺少 return · 編譯期發出 Warning ; 後端自動補 ret i32 0 防護網 (僅 int 回傳正確)
_Static_assert	_Static_assert(expr, "msg"); 條件於編譯期求值 · 不成立時輸出自訂訊息並中斷編譯 ; 不產生任何 LLVM IR

陳述式	說明
setjmp / longjmp	jmp_buf 型別變數儲存當前執行環境；setjmp() 設定跳轉點，longjmp() 跨函式非區域跳轉。可用於模擬例外處理與錯誤恢復
extern 宣告	extern int counter; 在全域層宣告外部符號；編譯器前端標記為外部符號，後端發射 external global 或 declare，跳過本地記憶體配置

4.4 函式

項目	說明
定義與呼叫	多參數、遞迴、互相遞迴
前置宣告	int foo(int, float); 先宣告再實作
回傳型別	int / float / double / char / long / void / 指標型別 / struct
陣列參數	傳入時自動退化為指標；支援二維 int arr[][4]、三維陣列參數
函式指標	宣告、typedef 別名、賦值、Callback、間接呼叫；全域與區域皆支援
main	int main() 與 int main(int argc, char *argv[])

4.5 預處理器

指令	說明
#define / #undef	常數巨集與函式型巨集（含 __VA_ARGS__ 與 ##__VA_ARGS__ 逗號吞噬）；支援以反斜線 \ 跨行接合的多行巨集，可與 # 字串化及條件編譯混用
條件編譯	#ifdef / #ifndef / #if (整數常數運算式) / #elif / #else / #endif
內建巨集	__LINE__ / __FILE__ / __func__ / __COUNTER__ / __DATE__ / __TIME__
預注入常數	INT_MAX/MIN、LONG_MAX/MIN、M_PI、M_E、INFINITY、NAN、NULL、EOF、TRUE/FALSE 等 30+ 個
修飾詞透傳	volatile / restrict 靜默承認並吸收；__attribute__((...)) 靜默忽略
其他	#pragma (忽略)、#error / #warning (印出訊息)、#line (忽略)、相鄰字串自動合併；__builtin_expect(expr,val)→(expr)、__builtin_unreachable()→(void)0、__builtin_constant_p(x)→0 靜默展開
#include	支援 #include "file.h" 與 #include <file.h>；前者搜尋相對路徑，後者靜默略過系統標頭（所有 libc 符號已預先注入）；支援 #ifndef include guard 保護

五、各功能詳細說明

5.1 基本要求功能

功能	支援範圍	特殊限制
整數 & 浮點四則 + - * /	int 與 float 四則運算；括號與優先權正確。純常數運算式在編譯期直接計算（常數折疊）	整數除法直接截斷，不四捨五入
比較運算	回傳 bool (i1)；int 與 float 相互比較時自動型別提升。支援隱式零擴充 (zext)，比較結果可作為整數直接參與算術運算	
if-then / if-then-else	任意巢狀；if(1) / if(0) 常數條件編譯期消除分支	
printf()	格式字串直接傳給 libc，所有 libc 支援的格式符號基本上都能用。float 引數自動提升為 double	格式字串必須是原始碼裡的字串字面值，不能是 char* 變數；需動態格式字串請改用 fprintf / sprintf
scanf()	格式字串直接傳給 libc，所有 libc 支援的格式符號基本上都能用	格式字串必須是字面值。引數只支援 &id (純量取址) 或 char 陣列名稱，不支援 &arr[i]、&p->x
自訂 ## 運算子	a ## b 計算 a^b + b^a，呼叫 myRuntime.c 的 my_hashhash。優先權與 * / 相同	兩個運算元請必須是 float

5.2 型別系統與資料結構

功能	支援範圍	特殊限制
double	宣告、四則運算、傳入函式、當回傳值；與 float 混算時自動提升	
bool / _Bool	值只有 0 或 1 (i1)；true / false 字面值；可用在條件與算術	
char	單一字元；算術前自動提升到 i32；char 陣列可存字串	char 是 8-bit；中文等多 byte 字元需用字串字面值
short / long / long long	short：i16；long / long long：i64；與 int 混算時自動提升	&、 、^、~ 對 long 結果不正確（硬編碼 i32）
unsigned 系列	unsigned char / short / int / long：除法、取餘數、使用無號指令	右移 >> 目前硬編碼 ashr i32，不區分有號/無號，Long 操作數會產生型別不匹配

功能	支援範圍	特殊限制
stdint.h 型別	int8_t ... int64_t、uint8_t ... uint64_t、size_t 等，不需 #include	
volatile / restrict	宣告時靜默承認，不影響編譯流程	只是吸收關鍵字，不產生 volatile 語意
一維陣列	int arr[5] = {1,2,3,4,5}; 可讀寫；傳入函式自動退化為指標	全域陣列不支援初始化清單
二維陣列	int a[3][4]; 讀寫 a[i][j]; 支援函式參數 int arr[][4]	全域陣列不支援初始化清單
三維陣列	int grid[2][3][4]; 支援宣告、讀寫 grid[i][j][k]、函式參數傳遞。後端使用 4 個索引的 GEP 指令精確計算記憶體步長	全域陣列不支援初始化清單
VLA	陣列大小由執行期變數決定：int buf[n];	只能在函式內宣告；大小受 stack 限制
struct (含位元欄位)	多欄位結構體；成員用 . 或 -> 存取。支援位元欄位宣告 (unsigned int flags : 4;)；後端自動處理多欄位壓縮、load/shift/mask/store 指令組合	不支援 == 直接比較兩個 struct
union	多欄位共用記憶體	
enum	列舉常數 (可手動指定值)；可在 switch / if / 算術中使用	底層 i32；enum Tag 不能作為變數宣告型別
typedef	型別別名，支援多層別名鏈。新增支援函式指標別名：typedef int (*Cmp)(int, int); 可用此別名宣告變數並呼叫	
具名初始化	C99 具名初始化語法。struct：struct Point p = {x=10, y=20}; 未指定欄位自動為 0。陣列：int arr[5] = {[2]=9, [4]=42}; 可跳躍指定索引，其餘自動為 0。 GNU 擴充範圍初始化：int arr[10] = {[0 ... 4]=1, [5 ... 9]=2}; 對連續索引批次賦值，底層展開為逐元素 store	
sizeof	sizeof(int)=4、sizeof(char)=1、sizeof(double)=8；編譯期常數	struct 含位元欄位時 sizeof 按壓縮後的整數大小計算
0x / 0b 字面值	0xFF、0b1010 可直接當整數使用	只能是整數字面值

功能	支援範圍	特殊限制
科學記號 / float 後綴	1.5e3、2.5f；LLVM IR 以 IEEE 754 十六進位精確表示	後綴 L (long double) 不支援
複合字面值	(Type){init} 語法；可在運算式內動態建立匿名 struct 或陣列並取得指標。例： func((Point){.x=1, .y=2});	匿名物件生命週期至函式結束
typeof(expr)	GNU C 擴充；推導運算式型別用於泛型巨集。 例：#define SWAP(a,b) do { typeof(a) t=a; a=b; b=t; } while(0) 以 Dry-Run 模式推導，不產生 IR 副作用	支援 typeof(expr) 與 typeof(TypeName) 等語法
_Alignof(type)	_Alignof(char)=1、_Alignof(int)=4、 _Alignof(double)=8；編譯期常數，與 sizeof 使用方式相同	
字串 / 字元跳脫序列擴充	新增支援十六進位 (\xNN) 與八進位 (\ooo) 跳脫序列，正確轉換為 LLVM IR 的 \XX 位元組表示法。支援：\n \t \r \ \" \' \0 \a \b \f \v \xNN \ooo	
陣列邊界防護 (Bounds Checking)	靜態：常數索引越界在編譯期發出 Warning (不中斷編譯)。動態：執行期索引越界自動觸發 abort() 並印出錯誤訊息，防止 Buffer Overflow。覆蓋一維、二維、三維陣列存取 (VLA 因大小未知，不做邊界防護)	
匿名 struct / union (Anonymous)	C11 標準巢狀匿名 struct / union 宣告。編譯器前端自動扁平化成員存取路徑，允許跨層直接存取內部欄位 (如 v.x、col.r)。後端處理 Little-Endian 架構下的記憶體共用與佈局配置	只支援完全匿名 (無欄位名) 的巢狀 struct/union；多層成員存取 (v.val.i) 不支援
_Static_assert	C11 編譯期靜態斷言。 _Static_assert(sizeof(long)==8, "msg"); 條件不成立時直接編譯錯誤，不產生任何 LLVM IR。常用於確認型別大小、對齊假設	條件為常數時純編譯期判斷；非常數條件不報錯，退化為執行期 abort() 呼叫
彈性陣列成員 (Flexible Array Member, C99)	struct 最後一個欄位宣告為無大小陣列：char data[]；。不佔 struct 本身的 sizeof 大小，需搭配 malloc(sizeof(struct X) + N) 動態配置。後端在 GEP 計算時處理偏移量	不能宣告含 FAM 的 struct 陣列

5.3 變數宣告與作用域

功能	支援範圍	特殊限制
多變數同行宣告	<code>int a=1, b=2, c;</code>	同行多個變數必須是相同型別
全域 / 區域變數 + 遮蔽	全域整個程式可用；區域只在 <code>{}</code> 內有效；內層同名變數遮蔽外層	全域變數初始值必須是編譯期常數；編譯器不主動攔截，傳入執行期運算式時 LLVM 會報錯
for 迴圈變數作用域隔離	<code>for(int i=0; i<n; i++)</code> 的 <code>i</code> 只在迴圈內存在 (C99)	
const 唯讀變數	<code>const int MAX = 100</code> ；目前不攔截寫入， <code>const</code> 僅作為常數折疊提示使用	透過指標間接寫入 <code>const</code> 不會被攔截
static 局部變數	函式內的 <code>static</code> 變數跨呼叫保留狀態；底層映射為 LLVM 內部全域變數	

5.4 控制流

功能	支援範圍	特殊限制
<code>while / do-while / for</code> (任意巢狀)	三種迴圈可任意混合巢狀； <code>break</code> 跳出最內層； <code>continue</code> 跳到下一次疊代	<code>do-while</code> 不做 LICM
<code>switch-case</code>	底層對應 LLVM <code>switch i32 · O(1)</code> 效率；支援 <code>fall-through</code> 與 <code>default</code>	條件只支援整數； <code>Long</code> 不支援；不支援 <code>float/double</code> ； <code>case</code> 值只能是數字字面值，不能使用 <code>enum</code> 常數名稱
巢狀 <code>if-else</code>	任意層次巢狀； <code>if(1) / if(0)</code> 編譯期消除	
<code>goto</code> 與標籤	含前向跳轉、後向跳轉、跳出多重巢狀迴圈	<code>goto</code> 不偵測是否跳過變數初始化宣告；違反時行為未定義 (C 標準限制，編譯器不攔截)

5.5 運算子

功能	支援範圍	特殊限制
三元運算子 <code>?:</code>	可巢狀；回傳型別依兩分支自動提升	兩個分支必須是相容型別，兩側為 <code>double</code> 時 <code>resultType</code> 判為 <code>Float</code>

功能	支援範圍	特殊限制
		(double/long 分支的提升行為可能不正確)
複合賦值 += -= *= /= %=	支援裸變數 (a += 3) 與 struct 成員 (p->x += 1) Ex: a &= b (複合賦值) → i32 或 i64 a & b (獨立表達式) → 固定 i32	解參考單純賦值 (*p = expr) 支援；複合賦值 (*p += 3) 不支援，需寫成 *p = *p + 3；不支援陣列元素複合賦值 (arr[i] += 1 → 需寫 arr[i] = arr[i] + 1)
前置 / 後置 ++ --	前置：先改再用；後置：先用再改；支援 int / float / double / 指標	只支援變數 ID；(*p)++、arr[i]++ 不支援
邏輯 && !	短路求值；以 Basic Block 跳轉實作	
位元運算 & ^ ~ << >>	完整支援整數位元運算；支援複合賦值	<< / >> / ~ 對 long 型別結果不正確 (硬編碼 i32)
逗號運算子	常見用途為 for 的 update (如 i++, j--)	

5.6 型別轉換

功能	支援範圍	特殊限制
隱式型別轉換	int 與 float 混算自動提升；float 賦值給 int 截斷小數；char 算術前提升為 i32；printf 的 float 引數自動升為 double	
顯式強制轉型	(int)、(float)、(double)、(char)、(long)、(struct X*)、(int)ptr 等	轉型不做越界保護

5.7 函式系統

功能	支援範圍	特殊限制
自訂函式 / 遞迴	多參數、遞迴、互相遞迴	
前置宣告	int foo(int, float); 先宣告再實作。具備嚴格簽名審查機制，實作時若回傳型別或參數不一致將於編譯期攔截報錯。	
陣列退化	傳入函式的陣列自動退化為指標；支援二維 (int	函式內無法知道陣列長度，需另外

功能	支援範圍	特殊限制
	arr[][4]) 與三維陣列參數	傳 n
函式指標	宣告 (int (*fp)(int,int)) 、typedef 別名 (typedef int (*Cmp)(int,int)) 、賦值、間接呼叫、Callback ; 全域與區域皆支援	
命令列參數	int main(int argc, char *argv[])	argv 的字串是唯讀的
setjmp / longjmp	jmp_buf 型別儲存當前執行環境快照 (stack frame 、register state) 。 setjmp(env) 設定跳轉點並回傳 0 ; longjmp(env, val) 跨函式跳回 setjmp 位置，此時 setjmp 回傳 val 。 可用於模擬 try/catch 例外處理與多層錯誤恢復	longjmp 跳回時，setjmp 所在函式必須仍在 call stack 上 (不能跳回已返回的函式)
尾呼叫優化 (TCO)	識別函式絕對尾端的自我遞迴呼叫 (return self(args)) 。 編譯器將最後一次遞迴呼叫替換為：把新引數 store 回參數的 alloca slots，再 br 回函式入口迴圈標籤。 將 O(N) 堆疊消耗壓縮為 O(1) ，解決深層遞迴的 Stack Overflow 問題	
分離編譯 (extern)	extern 全域變數：發射 @name = external global TYPE，由連結器從其他編譯單元解析位址。 extern 函式：發射 declare，與一般 libc 函式宣告相同機制。 同一個外部符號可多次 extern 宣告 (C 語意合法，去重複發射)	

5.8 指標與記憶體管理

功能	支援範圍	特殊限制
基本指標操作	int *p = &a; *p = 10; int v = *p; 支援各種型別指標	
指標算術	p + n 移動 n 個元素；*(p + n) 讀取；p2 - p1 指標相減 (回傳元素個數差，結果為 i32) (差值若超過 2 ³¹ 會截斷，大陣列請用 ptrdiff_t)	不支援 void* 算術 (會退化為 char* 行為，需先轉型為具體指標型別)
多重指標 int **	int **pp = &p; **pp = 5;	
箭頭運算子 ->	n->val 讀取與寫入	func()->val 不支援
動態記憶體 malloc /	配置與釋放；calloc 自動歸零；realloc 調整大小	malloc 宣告為 i32 大小參數，最

功能	支援範圍	特殊限制
free / calloc / realloc		大 2GB
memset / memcpy / memcmp memmove / memchr	記憶體區塊操作	
自我參考 struct (Linked List)	struct Node { int val; struct Node *next; } malloc 建鏈結串列	需手動 free
NULL 指標防護網	int *p = 0 或 NULL 自動轉換為 LLVM null	只保護「賦值 0 給指標」這個動作

5.9 預處理器

功能	支援範圍	特殊限制
#define 常數巨集	文字替換；#undef 取消；支援以反斜線 \ 跨行接合的多行巨集 (Object-like 與 Function-like)，可與 # 字串化及條件編譯混用	建議加括號 (#define N (1+2)) 避免優先權問題
函式型巨集	#define MAX(a,b) ((a)>(b)?(a):(b))	引數若有副作用 (如 MAX(i++, j)) 會展開兩次
條件編譯	#if / #elif 支援整數常數運算式與 defined()；#ifdef / #ifndef	#if 不支援浮點數比較
Token 連接 / 字串化	a##b 拼接 token；#x 轉為字串字面值	不完全支援巨集遞迴展開。 HASHHASH: '##' 這個 lexer token 是用於運算式層的自訂 ## 運算子 (5.1 節的 a ## b = a^b + b^a)，與預處理器層的 ## Token 連接是兩個完全不同的機制
__VA_ARGS__	#define LOG(fmt, ...) printf(fmt, __VA_ARGS__); ## __VA_ARGS__ 自動吞逗號	只能在函式型巨集中使用
動態內建巨集	__LINE__ / __FILE__ / __func__ / __COUNTER__ / __DATE__ / __TIME__	
預注入常數	INT_MAX/MIN、LONG_MAX/MIN、M_PI、M_E、INFINITY、NAN、NULL、EOF、TRUE/FALSE 等 30+ 個，不需 #include	

功能	支援範圍	特殊限制
#include	#include "file.h" 搜尋相對路徑並遞迴展開； #include <file.h> 靜默略過系統標頭（所有 libc 符號已預先注入，無需實際讀取）。 支援 #ifndef include guard 保護；#pragma once 被靜默忽略	系統標頭只是靜默略過，不做真正的路徑解析
其他靜默展開	#line N（靜默忽略）； __builtin_expect(expr,val)→(expr)、 __builtin_unreachable()→(void)0、 __builtin_constant_p(x)→0 靜默展開； __extension__ 靜默移除	

5.10 標準函式庫（對接 libc）

以下函式庫函式已在編譯器中宣告對應的 LLVM 外部符號，可直接使用，不需 #include。

功能	支援範圍	特殊限制
stdio.h	printf / scanf / sprintf / snprintf / fprintf / fscanf / sscanf / getchar / putchar /getc / fputc / ungetc / fopen / fclose / fread / fwrite / fseek / ftell / rewind / fgets / feof / ferror / perror / strerror / remove / rename / clearerr / setbuf / setvbuf / tmpfile / tmpnam / popen / pclose / vprintf / vfprintf / vsprintf / vsnprintf / puts / gets / fflush / freopen	printf / scanf 格式字串必須是字面值
string.h	strlen / strcpy / strncpy / strcat / strncat / strcmp / strncmp / strstr / strchr / strrchr / strtok / strdup / memset / memcpy / memcmp / memmove / memchr / strspn / strcspn / strpbrk / strsep / asprintf	strcpy / strcat 不限制長度，需自行確保 dst 夠大
stdlib.h	atoi / atof / abs / labs / llabs / strtol / strtoul / strtod / strtoll / strtod / malloc / free / calloc / realloc / qsort / bsearch / exit / _exit / getenv / putenv / system / atexit / rand / srand / aligned_alloc (i64 大小參數) / _Exit (立即終止，不執行 atexit) / div / ldiv	malloc 大小參數宣告為 i32；傳入 size_t/long 等 i64 型別時需先顯式轉型 (int)n
math.h	sqrt / pow / fabs / floor / ceil / fmod / sin / cos / tan / asin / acos / atan / atan2 / sinh / cosh / tanh / exp / exp2 / log / log2 / log10 / cbrt / hypot / lround / llround / nearbyint / round / trunc / fmin / fmax / fma / fdim / copysign / modf / frexp / ilogb / logb / ldexp / scalbn / isnan / isinf / isfinite / signbit	需連結 -lm（Makefile 已處理）
ctype.h	isdigit / isalpha / isalnum / isspace / isupper / islower / isprint / ispunct / isxdigit / isblank / iscntrl	

功能	支援範圍	特殊限制
	/ isgraph / toupper / tolower	
time.h	localtime / gmtime / mktime / strftime / difftime / time()	
signal.h	signal / raise	handler 內不能用 printf 等不可重入函式
assert.h	assert(條件)；條件為 false 時呼叫 abort() 中斷，不印出失敗位置	
setjmp.h	setjmp / longjmp / jmp_buf	

5.11 編譯期最佳化

功能	支援範圍	特殊限制
常數折疊	純常數算術在編譯期直接計算；支援 int / long / float / double	只有全部運算元都是常數才生效
代數化簡	$x+0 \rightarrow x$ ； $x*1 \rightarrow x$ ； $x*0 \rightarrow 0$ 等無效運算自動消除	
CSE 公共子式消除	CSE 的相同運算判斷以 tmp 名為 key，load 不做去重	每次進入新的 if / while / for body 都會 reset，不跨分支
DCE 死碼消除 (含逃逸分析)	UCE：不可達區塊移除；DSE：寫後從未讀的 store 移除；TDA：純計算結果未使用則移除；含指標逃逸分析	
LICM 迴圈不變式外提	while 和 for 內的迴圈不變式自動移到迴圈前執行	do-while 不做 LICM；含 call 的運算視為有副作用，不外提
強度削減	乘以 / 除以 2 的幕次 $\rightarrow$ shl / ashr / lshr；支援 i32 和 i64 動態判斷	
窺孔最佳化	局部 IR 模式識別與代數化簡	局部分析，不跨 Basic Block
常數分支壓平	if(1) / if(0) 編譯期直接消除分支	只對字面值常數生效
__builtin_expect	直接展開為 (expr)，val 提示值被忽略，不產生任何分支預測效果	
__attribute__	靜默吸收 GNU C 的 __attribute__((...)) 語法，不報錯	只是忽略，不產生任何效果
控制流防呆 (Missing Return)	非 void 函式若有執行路徑未包含 return 敘述，編譯期於 AST 走訪階段發出 Warning。後端自動補上 ret i32 0 防護網（僅 int 回傳正確；float / double / long 回傳型別的函式建議確保所有路徑都有 return）	只發出 Warning，不中斷編譯

功能	支援範圍	特殊限制
全域常數傳播 (GCP)	跨 Basic Block 追蹤常數賦值，將後續的變數讀取 (Load) 自動替換為常數，並觸發連鎖窺孔優化。具備逃逸與修改偵測機制 (MULTI / ESCAPE 標記)，確保優化安全性	CSE 只在同一 Basic Block 內；GCP 作用於整個函式 IR。若同一 alloca ptr 有多次不同值的 store (含迴圈內重複賦值) 標為 MULTI，或被當作函式呼叫的引數傳出則標為 ESCAPE，兩者皆停止傳播
LLVM 底層內建函式 (Intrinsics)	__builtin_clz(x)：計算前導零個數 (llvm.ctlz)。 __builtin_ctz(x)：計算尾端零個數 (llvm.cttz)。 __builtin_popcount(x)：計算 1 的位元數 (llvm.ctpop)。 __builtin_bswap32 / bswap64：位元組反轉 (llvm.bswap)。 直接對應 LLVM 底層指令，由硬體加速執行	引數必須是整數型別；clz / ctz 對 0 的行為依 LLVM 語意 (undef)
迴圈展開 (Loop Unrolling)	針對已知迭代次數的小型迴圈 (如 for(int i=0; i<4; i++))，編譯器前端自動展開迴圈體，直接消除 LLVM IR 層的條件判斷 (cond) 與跳轉 (br) 指令開銷。內建展開門檻防護 (防止 Code Bloat) 及動態 Step 計算	只對已知常數迭代次數的 for 迴圈生效；展開後指令數超過 128 條時不展開 (防止 Code Bloat)
尾呼叫優化 (TCO)	在 returnStatement 階段識別自我遞迴尾呼叫。將最後一次遞迴的 call 指令移除，改為 store 新引數 + br 回函式入口 tco_loop 標籤，消除 call/ret 的 stack frame 開銷	只支援直接自我遞迴；不發射 LLVM musttail 修飾符，而是以 store + br 迴圈實作

六、自訂 ## 運算子

本專案實作了題目要求的自訂運算子 ##，語意如下：

a ## b = a^b + b^a (a、b 都必須是 float；結果也是 float)

實作由 myRuntime.c 的 my_hashhash(float a, float b) 完成 (使用 powf)。編譯器在 multiplicativeExpression 規則中攔截 ## token，在 LLVM IR 產生 call float @my_hashhash(float, float)。

- 兩個運算元請必須是 float (後綴 f) (int 需先 (float) 轉型)
- ## 運算子要求兩個 operand 必須是 float 型別的變數。由於 C 語言浮點數 literal (如 1.5) 預設型別為 double，直接寫 1.5 ## 2.5 會報 type error。正確用法為先宣告 float 變數後再使用 ##，或著可使用後綴 f 精確定義。
- 結果是 float
- 寫法範例(可參見測試檔案):
 1. float magic_base;
 magic_base = 1.0f ## 5.0f;
 2. float ha;
 float hb;

```
float hr;
ha = 1.5;
hb = 2.5;
hr = ha ## hb;
```

七、不支援的功能

- `(*p)++`、`arr[i]++` 等複雜左值的 `++` / `--`
- `*p += 3`、`arr[i] += 1` 等複雜左值複合賦值 (需改寫為 `*p = *p + 3`)
- `printf` / `scanf` 以 `char*` 變數當格式字串 (改用 `fprintf` / `sprintf`)
- `scanf` 的複雜左值取址 (`&arr[i]`、`&p->x` ; 只支援 `&id`)
- `void*` 算術 (退化為 `char*` 行為)
- `typedef` 函式型別 (非指標形式，如 `typedef int func_t(int)`)
- `long double`、`__int128`
- 位元反 `~`、移位運算子 `<<` / `>>` 對 `long` 型別
- 巢狀函式定義
- 全域陣列的初始化清單
- C11 atomics、巨集遞迴展開
- `Long` 型別傳入 `switch` 時不報錯，但產生的 IR 型別不匹配，行為未定義
- 使用者自訂 `variadic` 函式內的 `va_arg`
- `calloc`/`realloc` 傳入 `size_t`/`long` 時靜默截斷為 `i32`，無報錯

八、轉換表格

轉換類別	範例語法 / 宣告 (Declaration)	目標型別 (Target)	來源型別 (Source)	LLVM IR
整數截斷 (Demotion)	<code>int8_t a = -128;</code>	<code>Char (i8)</code>	<code>Int (i32)</code>	<code>trunc i32 to i8</code>
	<code>uint8_t b = 255;</code>	<code>UnsignedChar (i8)</code>	<code>Int (i32)</code>	<code>trunc i32 to i8</code>
	<code>uint16_t d = 65535;</code>	<code>UnsignedShort (i16)</code>	<code>Int (i32)</code>	<code>trunc i32 to i16</code>
	<code>uint32_t f = 4294967295;</code>	<code>UnsignedInt (i32)</code>	<code>Long (i64)</code>	<code>trunc i64 to i32</code>
有號數擴展	<code>int64_t x64 = x32;</code>	<code>Long (i64)</code>	<code>Int (i32)</code>	<code>sext i32 to</code>

轉換類別	範例語法 / 宣告 (Declaration)	目標型別 (Target)	來源型別 (Source)	LLVM IR
(Sign Ext)				i64
	int res = a + 1; (<i>a 為 int8_t</i>)	Int (i32)	Char (i8)	sext i8 to i32
	int res = s + 1; (<i>s 為 short</i>)	Int (i32)	Short (i16)	sext i16 to i32
無號數擴展 (Zero Ext)	int res = b + 1; (<i>b 為 uint8_t</i>)	Int (i32)	UnsignedChar (i8)	zext i8 to i32
	printf("%d", (uint16_t)10);	Int (i32) (<i>Vararg</i>)	UnsignedShort (i16)	zext i16 to i32
布林型別轉換 (Bool ↔ Int)	bool flag = 5; (<i>非零為真</i>)	Bool (i1)	Int (i32)	icmp ne i32 %val, 0
	int val = flag;	Int (i32)	Bool (i1)	zext i1 to i32
浮點 ↔ 有號整數 轉換	float f = 10;	Float (float)	Int (i32)	sitofp i32 to float
	int i = 3.14f;	Int (i32)	Float (float)	fptosi float to i32
浮點 ↔ 無號整數 轉換	float f = (uint32_t)10;	Float (float)	UnsignedInt (i32)	uitofp i32 to float
	uint32_t u = 3.14f;	UnsignedInt (i32)	Float (float)	fptoui float to i32
浮點數精度升降	double d = 3.14f;	Double (double)	Float (float)	fpext float to double
	float f = 3.14;	Float (float)	Double (double)	fptrunc double to float
記憶體/指標位元 轉換 (Bitcast)	Node* n = malloc(8);	Struct Ptr (%Node*)	Void Ptr (i8*)	bitcast i8* to %Node*
	printf("%p", ptr);	Void Ptr (i8*)	Typed Ptr (i32*)	bitcast i32* to i8*